SIMATS ENGINEERING
ASSESSMENT TOOL 3

Name : E.Rohan
Reg No : 192524482

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Technical Presentation (Low)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Prepare a 5-slide presentation introducing DBMS: definition, objectives, advantages, and real-world applications.	BL2 – Understand	**5**
2	Present a comparison between file-based systems and DBMS using real-life case studies from banking or healthcare.	BL2 – Understand	**5**
3	Deliver a presentation explaining the three-level ANSI/SPARC architecture using diagrams and practical examples.	BL2 – Understand	**5**
4	Present the different data models (Hierarchical, Network, Relational) with diagrams and industry use cases.	BL1 – Remember	**5**
5	Create and present an ER diagram for an Online Library System, explaining each component clearly.	BL2 – Understand	**5**
6	Prepare a presentation on the EER model, demonstrating generalization and specialization with a University case study.	BL2 – Understand	**5**
7	Present the concept of data independence (logical and physical) and its significance in DBMS architecture.	BL2 – Understand	**5**

8	Deliver a technical presentation on the software components of a DBMS, explaining query processor, storage manager, and transaction manager.	BL1 – Remember	**5**
9	Prepare a presentation comparing strong and weak entities in ER modeling with real-world examples.	BL2 – Understand	**5**
10	Present a complete design walkthrough of an EER diagram for a Retail Chain Management System.	BL2 – Understand	**5**

Code of Conduct:

*I **E . R o h a n - 1 9 2 5 2 4 4 8 2** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E. Rohan

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals

Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Technical Presentation

Assessment Tool-3

CSA0501-Database Management System

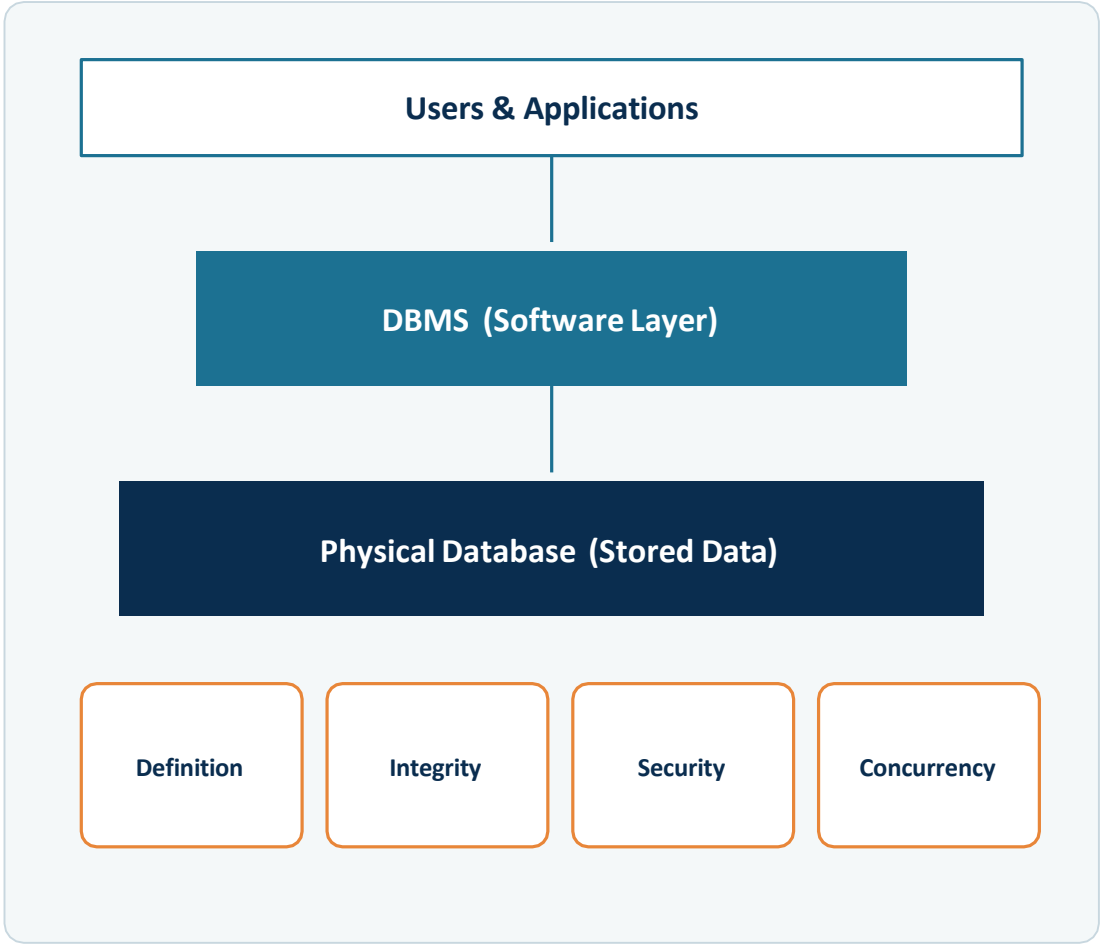
E . Rohan
192524482

What Is a DBMS — and Why Do We Need One?

A Database Management System (DBMS) is software that lets users define, create, populate, query, update and administer a database — acting as the controlled interface between raw stored data and every user or application that needs it.

Core Objectives

- ✦ Data independence — isolate applications from storage details
- ✦ Minimize redundancy while maximizing consistency
- ✦ Efficient access via indexing and query optimization
- ✦ Enforce integrity, security and access control
- ✦ Support concurrent multi-user access safely
- ✦ Provide backup, recovery and crash resilience



Advantages and Real-World Applications

Key Advantages

- Reduced data redundancy through centralized storage
- Consistent, integrity-checked data across the enterprise
- Controlled, concurrent data sharing among many users
- Fine-grained security and role-based access control
- Enforced standards and constraints on stored data
- Reliable backup and automated crash recovery
- Faster development via SQL instead of custom file code

Real-World Applications

- **Banking** – Secure transaction management
- **Airline Reservation** – Ticket booking & scheduling
- **Universities** – Student records management
- **E-Commerce** – Online sales & orders **Healthcare** – Patient information systems
- **Telecom** – Customer & billing management
- **Manufacturing / ERP** – Inventory & resource planning

Why Organizations Moved Away from File-Based Systems

Criteria	File-Based System	DBMS
Data Redundancy	High — duplicated across files	Minimized via centralized storage
Data Consistency	Hard to maintain, manual sync	Maintained through constraints
Data Sharing	Limited, program-specific access	Controlled concurrent multi-user access
Security	File-level, coarse-grained	Fine-grained, role-based access
Data Independence	None — programs tied to file layout	Logical & physical independence
Backup & Recovery	Manual, error-prone	Automated, transaction-based

Case Studies: Banking and Healthcare

Banking

Before: each branch kept separate customer files — duplicate, inconsistent records across branches.

With a DBMS: a centralized core-banking database gives real-time balance updates and shared customer data across every branch.

Why it matters: ACID transactions guarantee that a fund transfer either fully completes or fully rolls back, and cross-table joins support fraud detection.

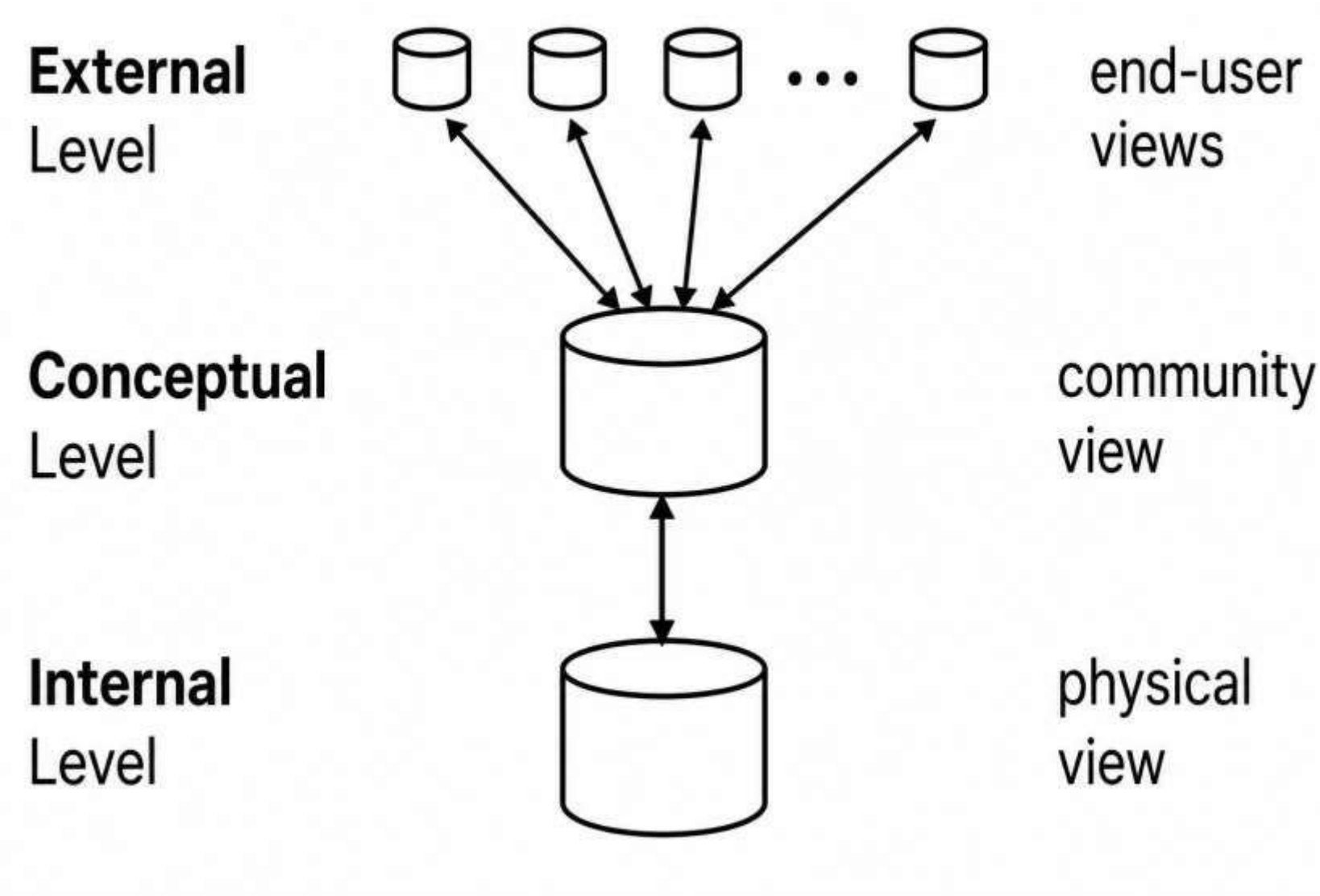
Healthcare

Before: paper or isolated files meant lost patient history and repeated diagnostic tests.

With a DBMS: an Electronic Health Record (EHR) system links demographics, diagnoses, prescriptions and lab results in one place.

Why it matters: controlled access with audit trails protects patient privacy while enabling faster, coordinated care across departments.

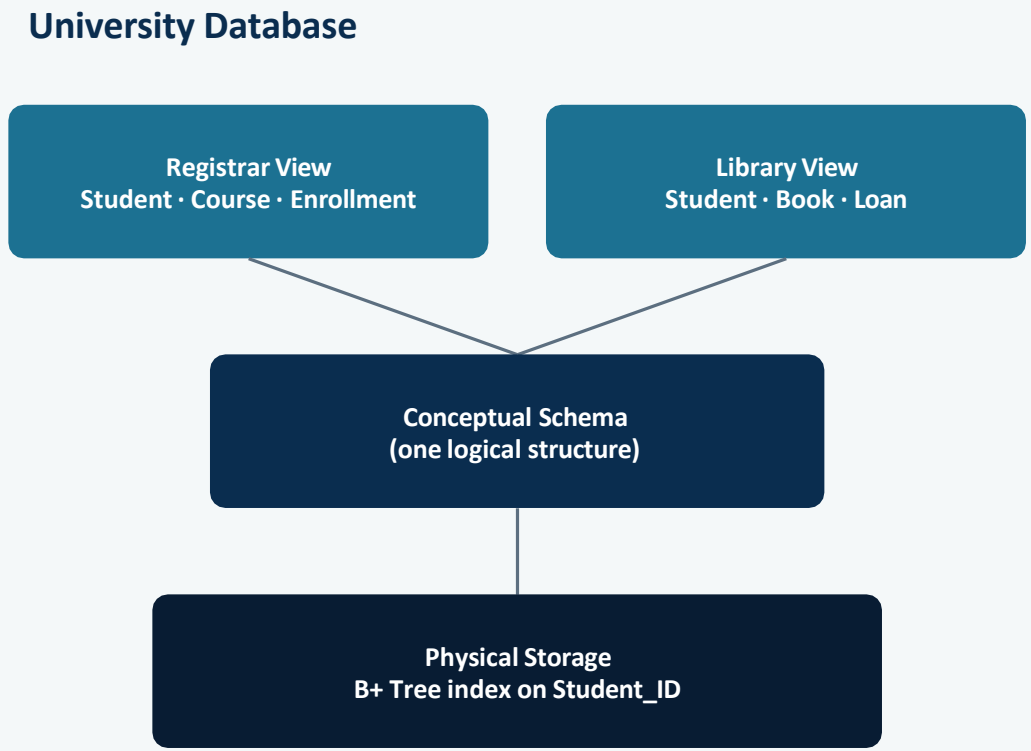
The Three-Level ANSI/SPARC Architecture



Mappings in Practice — A University Example

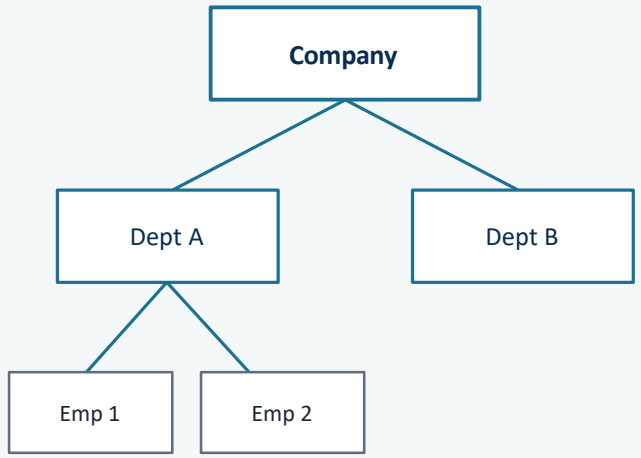
Why Two Mappings Matter

- External/Conceptual mapping links each user view to the single conceptual schema
- Conceptual/Internal mapping links that schema to physical storage
- Changing one level does not force changes in the others
- This separation is the basis of logical and physical data independence (see Q7)



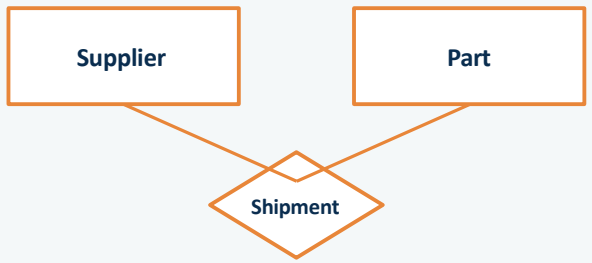
Hierarchical, Network and Relational Models

Hierarchical



One-to-many parent-child links only; fast to navigate, rigid to restructure.

Network



Records can have multiple owners — many-to-many sets via a link record.

Relational

ID	Name	Dept
S1	Asha	CSE
S2	Ravi	ECE
S3	Neha	CSE

Data stored as simple rows and columns linked by shared key values.

Where Each Model Is Used in Industry

Hierarchical

Industry use:
IBM IMS; legacy banking systems; XML document stores

Why: Fast when data is naturally tree-shaped, e.g. an org chart

Network

Industry use:
IDMS; early airline reservation and telecom systems

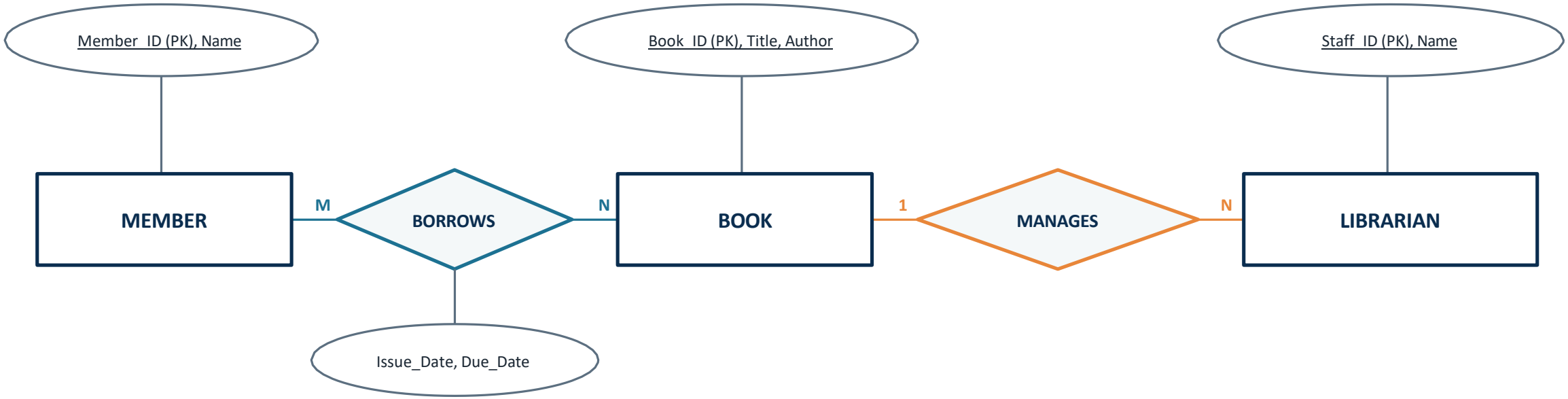
Why: Handles complex many-to-many links better than hierarchical

Relational

Industry use:
Oracle, MySQL, PostgreSQL, SQL Server

Why: Today's default: e-commerce, banking, ERP, CRM platforms

ER Diagram — Online Library System



A Member BORROWS many Books over time (M:N), recorded with Issue_Date and Due_Date. A LIBRARIAN MANAGES the Book catalogue (1:N) — one librarian oversees many book records.

Reading the Library ER Diagram

1 **Rectangles = Entities**

MEMBER, BOOK and LIBRARIAN are entity types — real-world objects the library tracks independently.

2 **Ellipses = Attributes**

Each entity carries descriptive attributes; underlined attributes (Member_ID, Book_ID, Staff_ID) are primary keys.

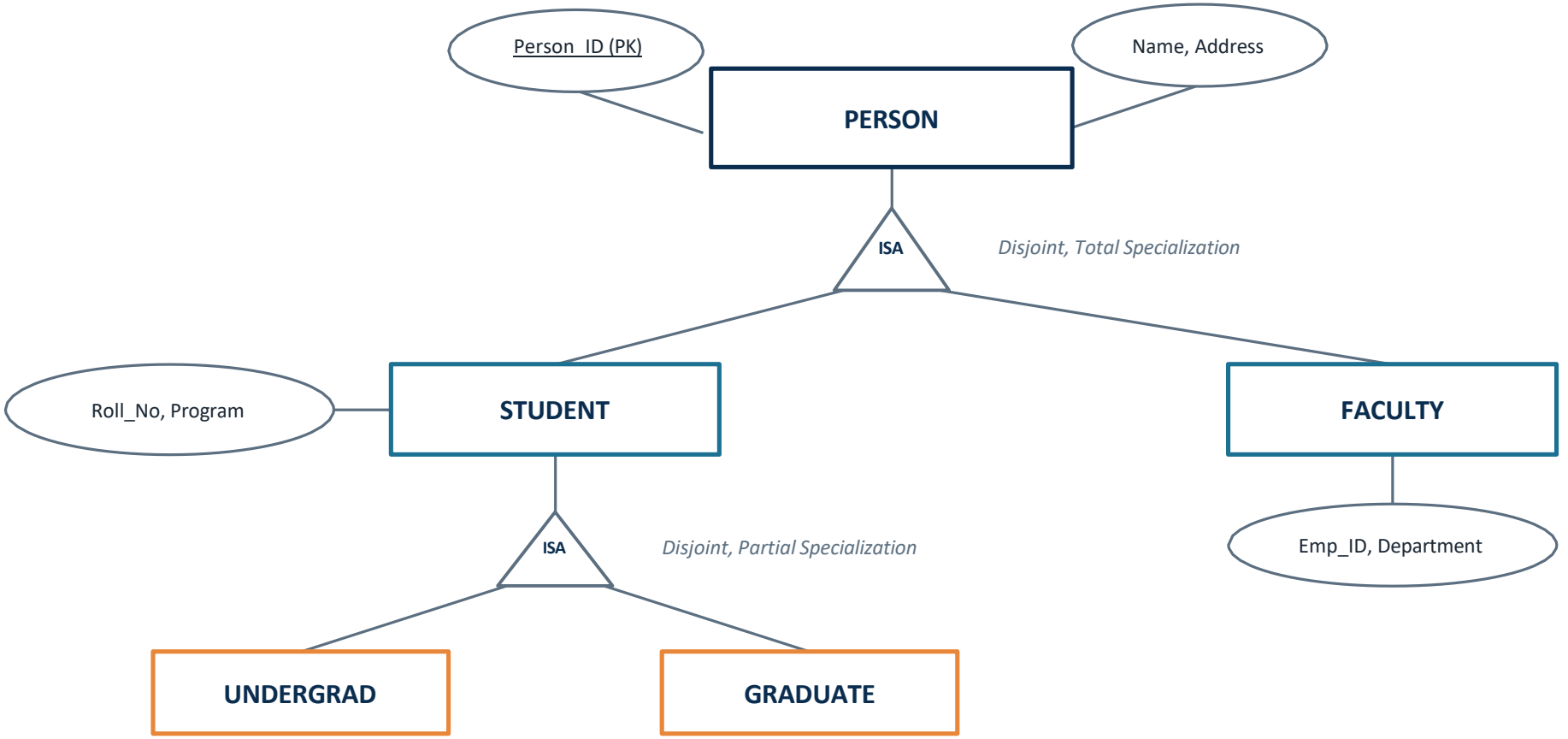
3 **Diamonds = Relationships**

BORROWS connects Member and Book; MANAGES connects Librarian and Book — each diamond names how entities interact.

4 **Lines = Cardinality**

BORROWS is many-to-many (a member borrows many books, a book is borrowed by many members over time); MANAGES is one-to-many.

EER Diagram — University: Generalization & Specialization



Why Generalize / Specialize the University Model?

Generalization (Bottom-Up)

- ✦ Start from STUDENT and FACULTY, notice shared attributes (ID, Name, Address)
- ✦ Factor those common attributes up into a PERSON superclass
- ✦ Removes duplicate attribute definitions and duplicate storage
- ✦ Shared relationships (e.g. Person has an Address) are defined once

Specialization (Top-Down)

- ✦ Start from PERSON and split it into meaningful roles: STUDENT, FACULTY
- ✦ Each subclass adds attributes only it needs (Roll_No vs Employee_ID)
- ✦ Total & disjoint here: every person is exactly one of the two roles
- ✦ STUDENT is specialized again into UNDERGRADUATE / GRADUATE for finer detail

Logical vs Physical Data Independence

Logical Data Independence

The ability to change the conceptual schema (e.g. add a new attribute or entity) without rewriting external views or application programs.

Example: adding an Email column to the Student table doesn't break existing queries that never reference it.

Physical Data Independence

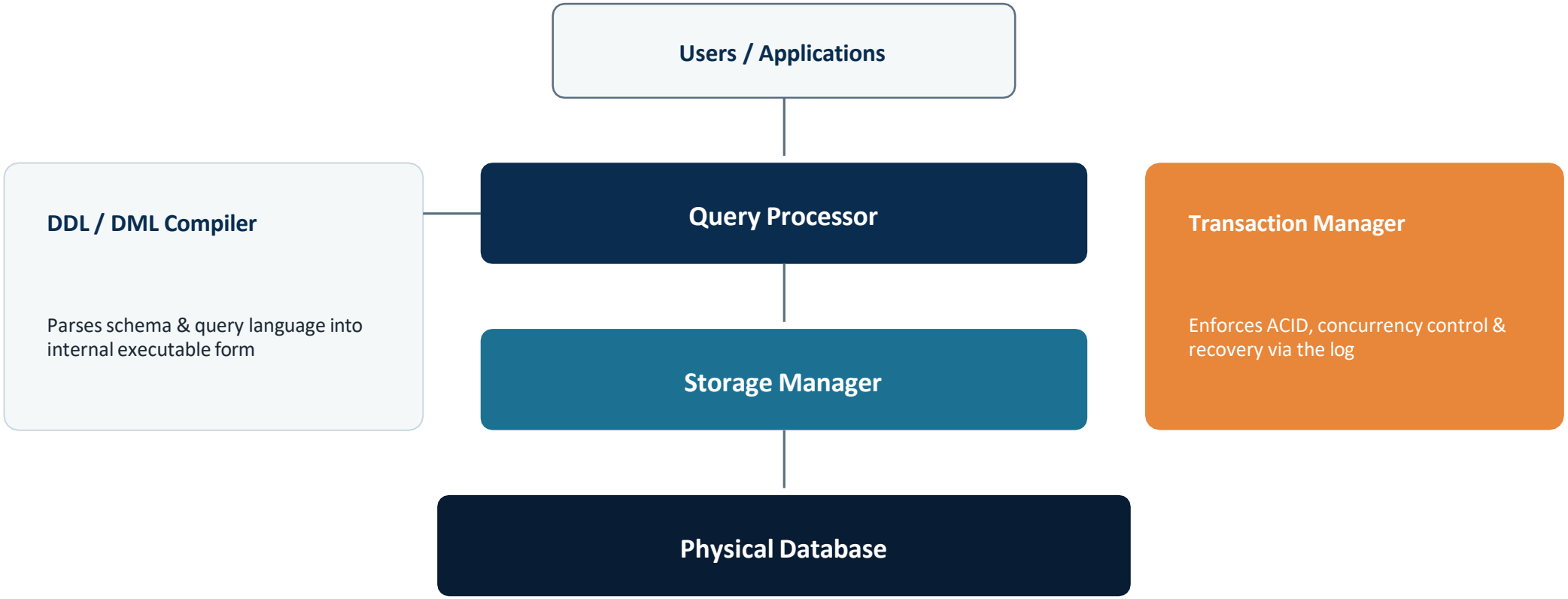
The ability to change the internal schema (storage structures, file organization, indexes) without changing the conceptual schema.

Example: switching Student storage from a sequential file to a B+ tree index doesn't affect any application logic.

Significance

- ✦ Enables schema evolution over time while cutting maintenance cost and isolating users from implementation detail — the very foundation the three-level ANSI/SPARC architecture (Q3) is built to provide.

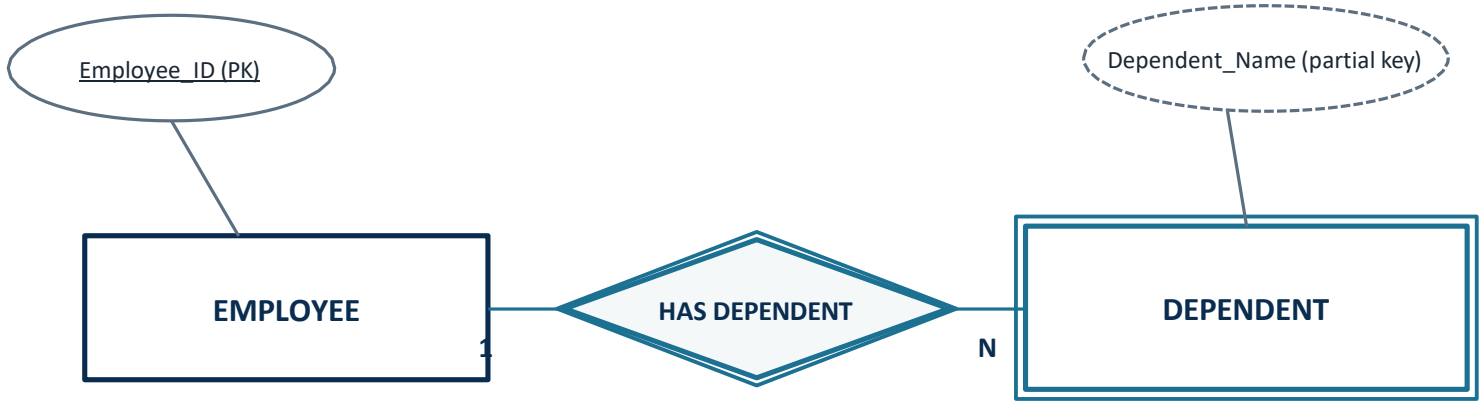
Inside the DBMS Engine



What Each Component Actually Does

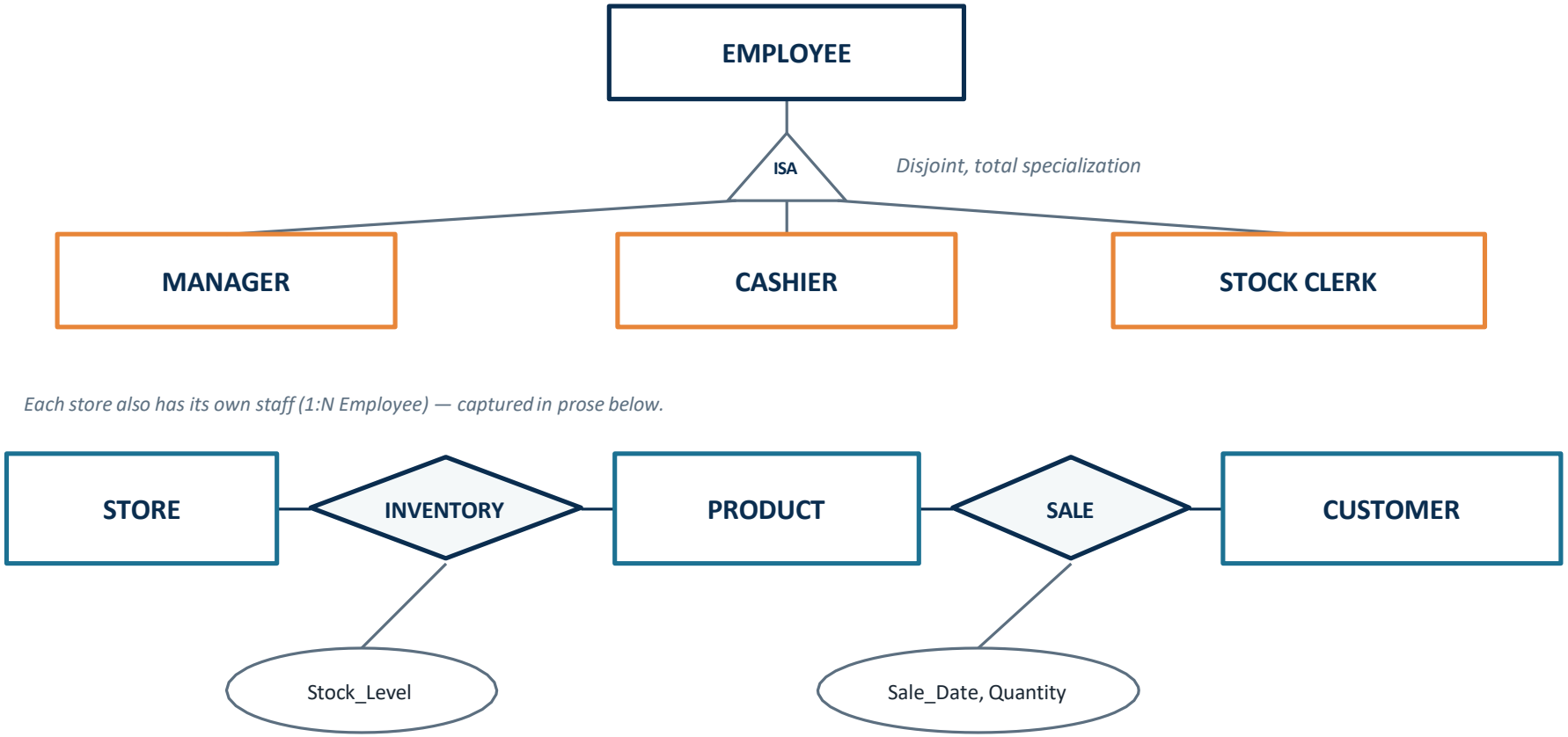
1	**Query Processor**	Parses incoming SQL for syntax and semantics, then the Query Optimizer chooses the most efficient execution plan (which index to use, join order, etc.) before handing it off for execution.
2	**Storage Manager**	Controls how data actually sits on disk: the File Manager organizes records, the Buffer Manager caches pages in memory, and index structures speed up retrieval of matching rows.
3	**Transaction Manager**	Guarantees ACID properties for every transaction — Atomicity, Consistency, Isolation, Durability — using locking or timestamping for concurrency control and the log for crash recovery.

Two Kinds of Entities in ER Modeling



	Strong Entity	Weak Entity
Notation	Single-line rectangle	Double-line rectangle
Primary Key	Has its own PK	No PK of its own — needs a partial key
Existence	Exists independently	Depends on an owner (identifying) entity
Example here	EMPLOYEE	DEPENDENT (identified through Employee_ID)

EER Diagram — Retail Chain Management System



EMPLOYEE specializes (disjoint, total) into MANAGER, CASHIER and STOCK CLERK; each STORE staffs many Employees (1:N). STORE stocks PRODUCT via INVENTORY (M:N, Stock_Level); CUSTOMER purchases PRODUCT via SALE (M:N), timestamped with Sale_Date and Quantity.

Design Walkthrough — Step by Step

1 Identify entities

STORE, EMPLOYEE, PRODUCT, CUSTOMER — the core objects the chain must track.

2 Model the role hierarchy

EMPLOYEE is generalized, then specialized (disjoint, total) into MANAGER, CASHIER, STOCK CLERK so role-specific attributes stay separate.

3 Define relationships & cardinality

STORE–PRODUCT via INVENTORY (M:N, stock level); CUSTOMER–PRODUCT via SALE (M:N, date & quantity); STORE–EMPLOYEE (1:N).

4 Assign keys & attributes

Give every entity a primary key (Store_ID, Product_ID, Customer_ID, Emp_ID) and the descriptive attributes it needs.

5 Resolve many-to-many links

Introduce associative attributes/entities on INVENTORY and SALE to carry facts that belong to the relationship itself, not to either entity alone.

6 Integrate into one EER schema

Combine every piece into a single enterprise-wide model that scales across multiple stores in the chain.